

SOLID Principles and Design Patterns in .NET

Name: Tushar Dev Singh

SRP: ReportGenerator handles report creation, ReportSaver handles saving.

OCP: IReportFormatter allows adding new formats (PDF, Excel) without modifying existing code.

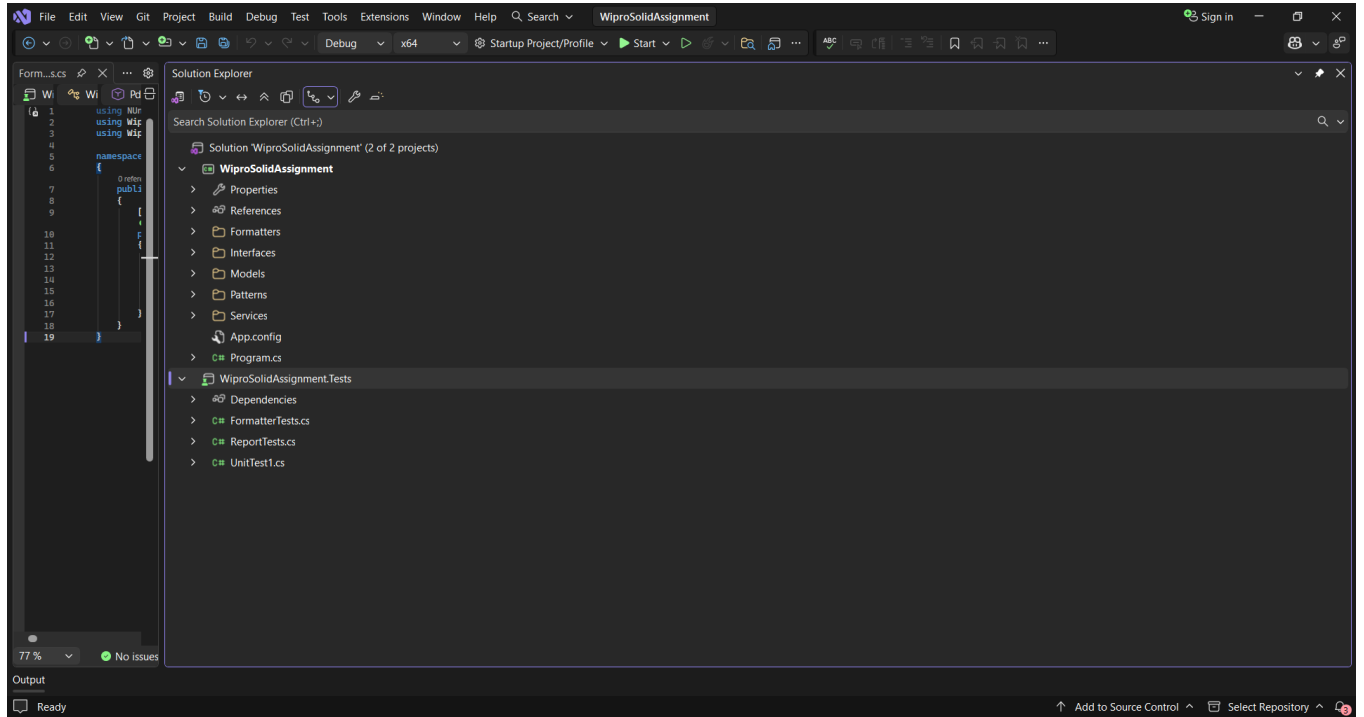
LSP: Derived report classes can replace base Report class without affecting behavior.

ISP: Interfaces are split so classes implement only required methods.

DIP: ReportService depends on abstractions using dependency injection.

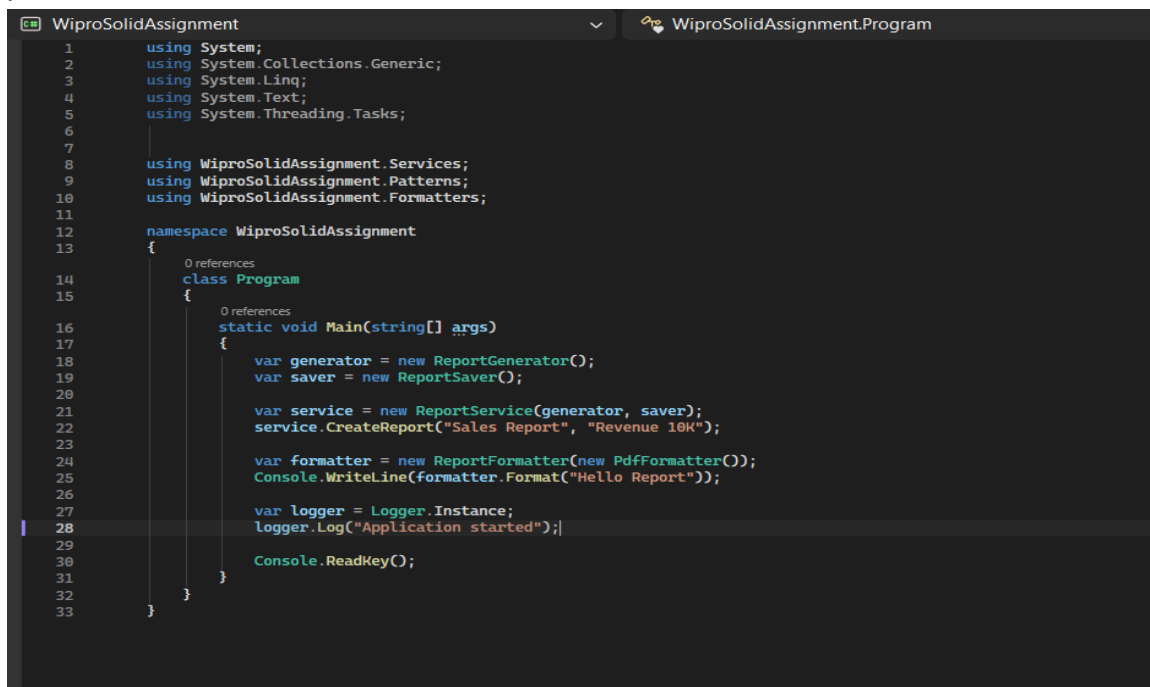
Design Pattern: Singleton Logger ensures only one instance exists globally.

Project Structure



Program.cs:

```
using System;
using System.Collections.Generic;
using WiproSolidAssignment.Services;
using WiproSolidAssignment.Patterns;
using WiproSolidAssignment.Formatters;
namespace WiproSolidAssignment
{
    class Program
    {
        static void Main(string[] args)
        {
            var generator = new ReportGenerator();
            var saver = new ReportSaver();
            var service = new ReportService(generator, saver);
            service.CreateReport("Sales Report", "Revenue 10K");
            var formatter = new ReportFormatter(new PdfFormatter());
            Console.WriteLine(formatter.Format("Hello Report"));
            var logger = Logger.Instance;
            logger.Log("Application started");
            Console.ReadKey();
        }
    }
}
```



```
WiproSolidAssignment
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7
8  using WiproSolidAssignment.Services;
9  using WiproSolidAssignment.Patterns;
10 using WiproSolidAssignment.Formatters;
11
12 namespace WiproSolidAssignment
13 {
14     0 references
15     class Program
16     {
17         0 references
18         static void Main(string[] args)
19         {
20             var generator = new ReportGenerator();
21             var saver = new ReportSaver();
22
23             var service = new ReportService(generator, saver);
24             service.CreateReport("Sales Report", "Revenue 10K");
25
26             var formatter = new ReportFormatter(new PdfFormatter());
27             Console.WriteLine(formatter.Format("Hello Report"));
28
29             var logger = Logger.Instance;
30             logger.Log("Application started");
31
32             Console.ReadKey();
33         }
34     }
35 }
```

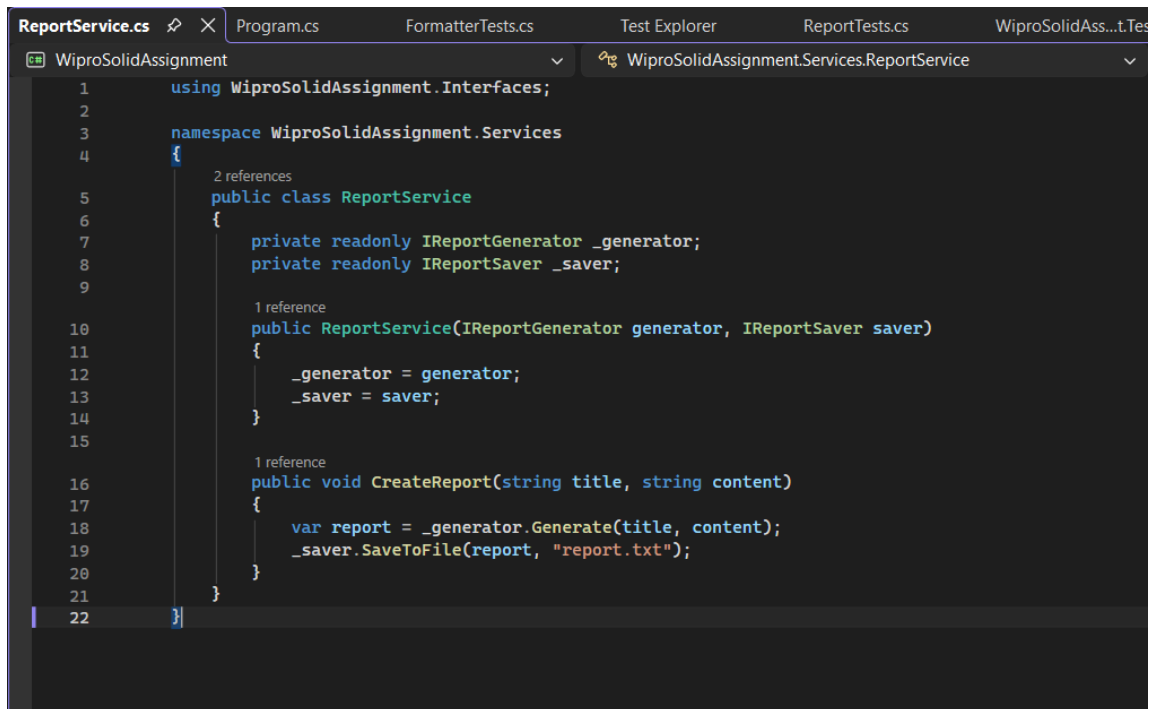
Logger.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System;
namespace WiproSolidAssignment.Patterns
{
    public sealed class Logger
    {
        private static Logger _instance;
        private static readonly object _lock = new object();
        private Logger() { }
        public static Logger Instance
        {
            get
            {
                lock (_lock)
                {
                    if (_instance == null)
                        _instance = new Logger();
                    return _instance;
                }
            }
        }
        public void Log(string message)
        {
            Console.WriteLine("LOG: " + message);
        }
    }
}
```

```
WiproSolidAssignment WiproSolidAssignment.Patterns.Logger
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6 using System;
7
8 namespace WiproSolidAssignment.Patterns
9 {
10     7 references
11     public sealed class Logger
12     {
13         private static Logger _instance;
14         private static readonly object _lock = new object();
15
16         1 reference
17         private Logger() { }
18
19         3 references 1/1 passing
20         public static Logger Instance
21         {
22             get
23             {
24                 lock (_lock)
25                 {
26                     if (_instance == null)
27                         _instance = new Logger();
28                     return _instance;
29                 }
30             }
31         }
32
33         1 reference
34         public void Log(string message)
35         {
36             Console.WriteLine("LOG: " + message);
37         }
38     }
39 }
```

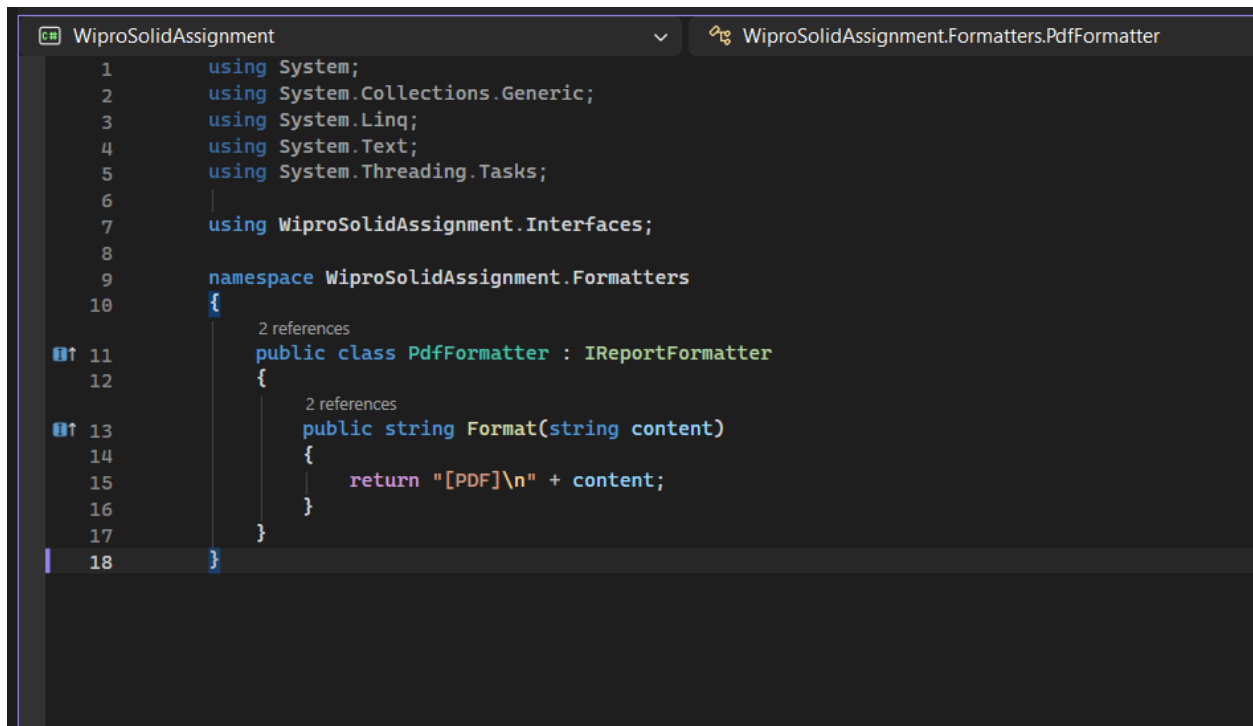
ReportService.cs:

```
using WiproSolidAssignment.Interfaces;
namespace WiproSolidAssignment.Services
{
    public class ReportService
    {
        private readonly IReportGenerator _generator;
        private readonly IReportSaver _saver;
        public ReportService(IReportGenerator generator, IReportSaver saver)
        {
            _generator = generator;
            _saver = saver;
        }
        public void CreateReport(string title, string content)
        {
            var report = _generator.Generate(title, content);
            _saver.SaveToFile(report, "report.txt");
        }
    }
}
```



PdfFormatter.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using WiproSolidAssignment.Interfaces;
namespace WiproSolidAssignment.Formatters
{
    public class PdfFormatter : IReportFormatter
    {
        public string Format(string content)
        {
            return "[PDF]\n" + content;
        }
    }
}
```



Output

A screenshot of a terminal window. The title bar at the top shows a file path: "D:\Wipro2026\Day3lab\Wiprc" with a close button on the right. The terminal content is as follows:

```
[PDF]  
Hello Report  
LOG: Application started  
|
```

The text is displayed in a light blue color on a black background. The cursor is positioned at the end of the fourth line, which is a single vertical bar.

Test Cases output:

The screenshot displays the Visual Studio IDE with the following components:

- Test Explorer:** Shows the test run results for `WiproSolidAssignment.Tests (3)`. The test run finished with 3 tests passed, 0 failed, and 0 skipped, taking 177 ms. The test results are as follows:

Test	Duration	Traits	Error Message
WiproSolidAssignment.Tests (3)	8 ms		
WiproSolidAssignment.Tests (3)	8 ms		
FormatterTests (1)	5 ms		
LoggerTests (1)	1 ms		
ReportTests (1)	2 ms		
- Solution Explorer:** Shows the project structure for `WiproSolidAssignment`. The project contains several files and folders, including `WiproSolidAssignment`, `WiproSolidAssignment.Tests`, `Interfaces`, `Models`, `Patterns`, `Services`, `App.config`, `Program.cs`, and `Dependencies`.
- Output:** Shows the test results for `WiproSolidAssignment.Tests (3)`. The output indicates that the test run finished with 3 tests passed, 0 failed, and 0 skipped, taking 177 ms.